do live object detection using Google Colab on a Chromebook without enabling Linux.

Because Google Colab runs entirely in your web browser, you do not need to turn on the Chromebook's Linux environment to write code or train models.

How It Works Without Linux

Google Colab runs on remote Google servers, meaning it cannot automatically see your physical Chromebook webcam. To get around this, you must use a small piece of JavaScript code inside your Colab notebook. This code bridges the gap by safely requesting access to your Chromebook webcam through the browser, capturing the video frame-by-frame, and sending those images directly to your Python object detection model.

Complete Step-by-Step Code

You can copy and paste the following blocks of code directly into a blank Google Colab notebook to start detecting objects right away using a pre-trained MobileNet SSD model.

Step 1: Import Libraries

Create a new code cell and paste this code to import the required tools:

python

```
import cv2
import numpy as np
from IPython.display import display, Javascript, Image
from google.colab.output import eval_js
import base64
```

Use code with caution.

Step 2: Set Up the Web-to-Python Camera Bridge

Paste this JavaScript wrapper into a new cell. This is the crucial step that bypasses the need for Linux by letting your browser stream the live webcam feed directly into Colab:

python

```
def video_stream():
  js = Javascript('''
    var video;
    var div = null;
    var stream;
    var captureCanvas;

    async function initCamera() {
      if (div !== null) return;
      div = document.createElement('div');
      div.style.border = '2px solid black';
      div.style.padding = '4px';
      div.style.width = '640px';

      video = document.createElement('video');
      video.style.display = 'block';
      video.width = 640;
      video.height = 480;
      video.autoplay = true;
      video.playsInline = true;

      stream = await navigator.mediaDevices.getUserMedia({video: true});
      video.srcObject = stream;
      div.appendChild(video);
      document.body.appendChild(div);

      captureCanvas = document.createElement('canvas');
      captureCanvas.width = 640;
      captureCanvas.height = 480;
    }

    async function captureFrame() {
      await initCamera();
      var ctx = captureCanvas.getContext('2d');
      ctx.drawImage(video, 0, 0, 640, 480);
      return captureCanvas.toDataURL('image/jpeg', 0.8);
    }

    window.captureFrame = captureFrame;
    ''')
  display(js)

def js_to_image(js_reply):
  image_bytes = base64.b64decode(js_reply.split(',')[1])
  jpg_as_np = np.frombuffer(image_bytes, dtype=np.uint8)
  img = cv2.imdecode(jpg_as_np, flags=1)
  return img
```

Use code with caution.

Step 3: Run the Live Detection Loop

This code downloads a lightweight object detector, triggers your webcam via the browser, and continuously processes the frames to draw bounding boxes around everyday items like people, chairs, or bottles:

python

```
# Download a lightweight pre-trained model
!wget --no-check-certificate https://githubusercontent.com
!wget --no-check-certificate https://githubusercontent.com

# Load the network
net = cv2.dnn.readNetFromCaffe('deploy.prototxt',
'mobilenet_iter_73000.caffemodel')
CLASSES = ["background", "aeroplane", "bicycle", "bird", "boat", "bottle",
"bus", "car", "cat", "chair", "cow", "diningtable", "dog", "horse",
"motorbike", "person", "pottedplant", "sheep", "sofa", "train",
"tvmonitor"]

# Start webcam stream
video_stream()

# Infinite loop to detect objects frame by frame
while True:
  # Get frame from Chromebook webcam
  js_reply = eval_js('captureFrame()')
  if not js_reply:
    break

  frame = js_to_image(js_reply)
  (h, w) = frame.shape[:2]

  # Run object detection
  blob = cv2.dnn.blobFromImage(cv2.resize(frame, (300, 300)), 0.007843,
(300, 300), 127.5)
  net.setInput(blob)
  detections = net.forward()

  # Draw boxes on detected objects
  for i in np.arange(0, detections.shape[2]):
    confidence = detections[0, 0, i, 2]
    if confidence > 0.5: # 50% confidence threshold
```

```
      idx = int(detections[0, 0, i, 1])
      box = detections[0, 0, i, 3:7] * np.array([w, h, w, h])
      (startX, startY, endX, endY) = box.astype("int")

      label = f"{CLASSES[idx]}: {confidence * 100:.2f}%"
      cv2.rectangle(frame, (startX, startY), (endX, endY), (0, 255, 0), 2)
      cv2.putText(frame, label, (startX, startY - 15),
cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

  # Update the display output in Colab
  _, encoded_img = cv2.imencode('.jpg', frame)

  display(Image(data=encoded_img), display_id='live_feed')
```